Assignment -05 _HPC_LAB_U23AI0

Program_01:

Program_02:

02_p_.c

```c
#include <stdio.h>
#include <omp.h>

int main() {
    int n = 100;
    int arr[n];

    // Initialization
    for (int i = 0; i < n; i++) {
        arr[i] = i + 1;
    }

    long long global_sum = 0;
    double global_product = 1.0;

    // Parallel Region
    #pragma omp parallel
    {
        long long local_sum = 0;
        double local_product = 1.0;

        // Divide work
        #pragma omp for
        for (int i = 0; i < n; i++) {
            local_sum += arr[i];
            local_product *= arr[i];
        }

        // Named critical for sum
        #pragma omp critical(global_sum_section)
        {
            global_sum += local_sum;
        }

        // Named critical for product
        #pragma omp critical(global_product_section)
        {
            global_product *= local_product;
        }
    }

    // Output
    printf("Global Sum     : %lld\n", global_sum);
    printf("Global Product : %e\n", global_product);

    return 0;
}
```

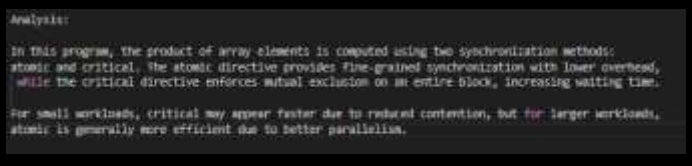
output:

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/06_lab_$ ./q1
Log Product (Atomic)   : 957498.348564
Time (Atomic)          : 0.094944 seconds
Log Product (Critical) : 957498.348564
Time (Critical)        : 0.283989 seconds
```

Output:

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/06_lab_$ ./p2
Global Sum     : 5050
Global Product : 9.332622e+157
```

Analysis:



Analysis:

Analysis:
his program demonstrates the use of multiple named critical sections to update independent shared variables (sum and product). By assigning different names to critical sections, separate locks are created, allowing threads to update sum and product concurrently without unnecessary blocking. This improves parallel efficiency compared to using a single unnamed critical section.

Program_03:

03_p_.c

```c
#include <stdio.h>
#include <omp.h>

#define N 100
#define BUCKETS 4

int main() {
    int arr[N];
    for (int i = 0; i < N; i++) arr[i] = i + 1;

    // ---------- Global Lock ----------
    omp_lock_t lock;
    omp_init_lock(&lock);

    long long sum_global = 0;
    double start = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        omp_set_lock(&lock);
        sum_global += arr[i];
        omp_unset_lock(&lock);
    }

    double time_global = omp_get_wtime() - start;
    omp_destroy_lock(&lock);

    // ---------- Bucket Lock ----------
    omp_lock_t bucket_locks[BUCKETS];
    long long bucket_sum[BUCKETS] = {0};

    for (int i = 0; i < BUCKETS; i++)
        omp_init_lock(&bucket_locks[i]);

    start = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        int b = i % BUCKETS;

        omp_set_lock(&bucket_locks[b]);
        bucket_sum[b] += arr[i];
        omp_unset_lock(&bucket_locks[b]);
    }

    long long sum_bucket = 0;
    for (int i = 0; i < BUCKETS; i++) {
        sum_bucket += bucket_sum[i];
        omp_destroy_lock(&bucket_locks[i]);
    }

    double time_bucket = omp_get_wtime() - start;

    printf("Global Lock Sum   : %lld, Time: %f\n", sum_global, time_global);
    printf("Bucket Lock Sum   : %lld, Time: %f\n", sum_bucket, time_bucket);

    return 0;
}
```

Output :

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/06_lab_$ ./p3
Global Lock Sum   : 5050, Time: 0.012516
Bucket Lock Sum   : 5050, Time: 0.006561
```

Analysis:

Analysis:

Global lock causes high contention as all threads compete for a single lock, reducing parallelism. Bucket locking reduces contention by allowing multiple threads to work on different partitions simultaneously. However, managing multiple locks introduces slight overhead. Overall, bucket locking improves performance for larger workloads.

Program_04:

04_p_.c

```c
#include <stdio.h>
#include <omp.h>

#define N 4

int main() {
    int A[N][N], x[N], y[N] = {0};

    // Initialization (small values)
    for (int i = 0; i < N; i++) {
        x[i] = i + 1;
        for (int j = 0; j < N; j++) {
            A[i][j] = j + 1;
        }
    }

    #pragma omp parallel
    {
        #pragma omp single
        {
            for (int i = 0; i < N; i++) {
                #pragma omp task depend(out: y[i])
                {
                    for (int j = 0; j < N; j++) {
                        y[i] += A[i][j] * x[j];
                    }
                }
            }
        }
    }

    printf("Result vector y:\n");
    for (int i = 0; i < N; i++) {
        printf("%d ", y[i]);
    }
    printf("\n");

    return 0;
}
```

Output

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/06_lab_$ ./p4
Result vector y:
30 30 30 30
```

Analysis:

Analysis:

This program performs a matrix-vector multiplication using OpenMP tasks with dependencies. Each task computes one element of the result vector y, and the depend clause ensures that tasks are executed in the correct order. The single directive ensures that only one thread creates the tasks, while the parallel region allows multiple threads to execute them concurrently.

Analysis _02 :

The matrix-vector multiplication is decomposed into tasks, where each task computes one row of the result. The depend clause ensures proper data flow and avoids race conditions. This allows efficient parallel execution while maintaining correctness.

Program_05:

05_p_.c

```c
#include <stdio.h>
#include <omp.h>

#define N 400

int main() {
    int A[N], B[N];


    for (int i = 0; i < N; i++) {
        A[i] = i + 1;
        B[i] = (i % 10) + 1;
    }

    long long sum = 0, diff = 0;
    double product = 1.0, division = 0.0;

    #pragma omp parallel
    {
        #pragma omp single
        {

            #pragma omp task
            {
                long long local_sum = 0;
                for (int i = 0; i < N; i++)
                    local_sum += A[i] + B[i];

                #pragma omp critical
                sum += local_sum;
            }


            #pragma omp task
            {
                long long local_diff = 0;
                for (int i = 0; i < N; i++)
                    local_diff += (A[i] - B[i]);

                #pragma omp critical
                diff += local_diff;
            }

            /
            #pragma omp task
            {
                double local_product = 1.0;
                for (int i = 0; i < 20; i++)
                    local_product *= (A[i] * B[i]);

                #pragma omp critical
                product *= local_product;
            }


            #pragma omp task
            {
                double local_div = 0.0;
                for (int i = 0; i < N; i++)
                    local_div += (double)A[i] / B[i];

                #pragma omp critical
                division += local_div;
            }


            #pragma omp taskwait


            printf("Sum        : %lld\n", sum);
            printf("Difference : %lld\n", diff);
            printf("Product    : %e\n", product);
            printf("Division   : %f\n", division);
        }
    }

    return 0;
}
```

Output:

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/06_lab_$ ./p5
Sum        : 82400
Difference : 78000
Product    : 3.203691e+31
Division   : 23245.952381
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/06_lab_$
```

Analysis:

Analysis:
The program uses multiple tasks to perform independent computations in parallel. Race conditions are avoided using local variables and critical sections. Task synchronization is ensured using taskwait, guaranteeing that results are printed only after all tasks complete.